Generating All Triangulations of Biconnected Plane Graphs in Amortized Constant Time Per Triangulation

Tawkir Aziz Rahman
Department of Computer Science and Engineering
Bangladesh University of Engineering and Technology (BUET)
Dhaka-1000, Bangladesh

June 2, 2026

Abstract

We present an algorithm for generating all triangulations of a biconnected plane graph G with n vertices in $O(1)$ amortized time per triangulation and $O(n)$ space. Our algorithm establishes a double-layer tree structure consisting of a recursive tree of faces and, for each face, a local genealogical tree of triangulations. The key innovation is a novel "safe root vertex" selection strategy that prevents the multi-edge problem inherent in biconnected graphs, thereby extending the optimal complexity previously achievable only for triconnected graphs. We prove that every face contains at least two safe root vertices regardless of prior triangulations, and we show how to find one in $O(n)$ time using a two-pointer technique. A dynamic global chord set together with a valid generating set (VGS) allows constant-time edge flips and safe pruning of invalid branches. Extensive experiments on 35 diverse test cases validate the theoretical bounds: the algorithm generates tens of millions of triangulations while maintaining nanosecond-level per-triangulation time and linear memory usage. To the best of our knowledge, this is the first algorithm for generating all triangulations of a biconnected plane graph with $O(1)$ amortized time per triangulation.

Acknowledgements

I express my sincere gratitude to my supervisor for his invaluable guidance and encouragement throughout this research. I also thank the authors of the foundational work on triconnected plane graph triangulations, Mohammad Tanvir Parvez, Md. Saidur Rahman, and Shin-ichi Nakano, whose elegant algorithm provided the starting point for this investigation. Finally, I am grateful to the Department of Computer Science and Engineering, BUET, for providing the necessary resources and a conducive research environment.

Contents

List of Figures

List of Algorithms

Chapter 1

Introduction

1.1 Motivation and Applications

The problem of generating all triangulations of plane graphs arises in many areas of computational geometry [**?**], VLSI floorplanning [**?**], and graph drawing [**?**]. A triangulation of a plane graph is a maximal set of non-crossing chords that partition each face into triangles. Enumerating all possible triangulations is essential for exploring geometric configurations, optimizing layouts, and testing hypotheses about graph properties.

In VLSI design, for instance, different triangulations of a floorplan correspond to different routing possibilities. In mesh generation, enumerating triangulations helps to find optimal finite element meshes. In graph drawing, triangulations are used to create straight-line embeddings and to study graph properties.

1.2 Challenges in Generating All Triangulations

The number of triangulations of a planar graph can be exponential in the number of vertices. For a convex polygon with n vertices, the number of triangulations is the Catalan number C_{n-2}, which grows as $\Theta(4^n n^{-3/2})$. For general plane graphs, the count can be even larger. Therefore, any algorithm that lists all triangulations must handle an exponential output size.

The main challenges in designing such algorithms are threefold:

1. **Exponential output:** The algorithm must run in time proportional to the number of triangulations; otherwise it would be inefficient.

2. **Avoiding duplicates:** Since the output is huge, storing previously generated triangulations to check for duplicates would require exponential space and time.

3. **Space efficiency:** The algorithm should use only polynomial space (preferably linear in the input size) and not store all outputs simultaneously.

Classical approaches to generating combinatorial objects include orderly algorithms [?] and reverse search [?]. Orderly algorithms generate objects in a canonical form and output only those that are canonical, thus avoiding duplicates without storing all previous objects. Reverse search defines a spanning tree on the graph of objects and traverses it using a parent function, again without storing the entire set.

1.3 Previous Work

There is a rich literature on triangulation enumeration. For convex polygons, it is well known that triangulations correspond to binary trees, and efficient generation algorithms exist that run in constant amortized time per triangulation [?].

For planar graphs, the problem is more challenging. Li and Nakano [?] gave an algorithm to generate all biconnected plane triangulations with at most n vertices in $O(r^2 n)$ time per triangulation, where r is the number of vertices on the outer face. Later, Nakano improved this to $O(rn)$ time per triangulation.

The most relevant prior work is the algorithm by Parvez, Rahman, and Nakano [?], which generates all triangulations of a *triconnected* plane graph in $O(1)$ amortized time per triangulation using $O(n)$ space. This algorithm defines a genealogical tree of triangulations where each node is a triangulation and each edge corresponds to a chord flip. By traversing this tree in a depth-first manner, all triangulations are generated without duplicates.

However, the algorithm of Parvez et al. is limited to triconnected graphs. As we will explain in Section 1.5, extending it directly to biconnected graphs leads to the multi-edge problem caused by separation pairs. This limitation is the main motivation for our work.

1.4 The Parvez–Rahman–Nakano 2011 Algorithm (Overview)

We briefly recall the key ideas of the algorithm for triconnected plane graphs, as they form the foundation of our approach.

Given a triconnected plane graph G with n vertices, the algorithm processes all faces simultaneously. For each face F_i (a cycle), it constructs a genealogical tree of triangulations. The root of the tree is a *fan triangulation* obtained by picking an arbitrary vertex as the root and connecting it to all non-neighbouring vertices on the face. From the root, child triangulations are generated by flipping a *generating chord* (a chord that, when flipped, yields a new triangulation whose leftmost blocking chord is the flipped chord). The parent of any non-root triangulation is obtained by flipping its leftmost blocking chord. This parent–child relationship is unique, and the tree contains every triangulation exactly once.

The algorithm runs in $O(1)$ amortized time per triangulation and uses $O(n)$ space. The key properties that enable this efficiency are:

- Flipping a chord takes constant time.
- The depth of the genealogical tree is $O(n)$.
- The number of generating chords decreases along any path from the root.

1.5 Limitation of Prior Work: The Multi-Edge Problem in Biconnected Graphs

The Parvez–Rahman–Nakano algorithm works only for triconnected plane graphs. The reason is that in triconnected graphs, no separation pair exists, so flipping chords never creates multi-edges. In biconnected graphs, however, separation pairs are possible, and independent triangulation of different faces can lead to the same chord being added twice.

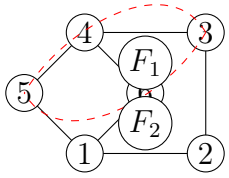


Figure 1.1: A biconnected plane graph with a separation pair $(3, 5)$. Triangulating faces F_1 and F_2 independently can add the chord $(3, 5)$ twice, creating a multi-edge.

Figure 1.1 illustrates a biconnected graph where vertices 3 and 5 form a separation pair: removing them disconnects the graph. If we triangulate the outer face F_1 and the inner face F_2 independently using the algorithm of Parvez et al., we might add the chord $(3, 5)$ in both faces, leading to a multi-edge. Such multi-edges are not allowed in simple graphs, and the algorithm would generate invalid triangulations.

Thus, a direct application of the Parvez–Rahman–Nakano algorithm to biconnected graphs fails. Our goal is to overcome this limitation while preserving the optimal $O(1)$ amortized time per triangulation.

1.6 Our Contribution

We present the first algorithm for generating all triangulations of a *biconnected* plane graph in $O(1)$ amortized time per triangulation and $O(n)$ space. Our main innovations are:

1. **Local genealogical trees:** Instead of processing all faces simultaneously, we process faces one by one in a fixed order. For each face, we construct a local genealogical tree of triangulations that are compatible with previously triangulated faces.

2. **Safe root vertex selection:** We introduce the concept of a *safe root vertex* for a face. A safe root vertex guarantees that the fan triangulation from that vertex does not create any multi-edge with chords from previous faces. We prove that every face in a biconnected plane graph contains at least two safe root vertices, regardless of the chords already present.

3. **Linear-time safe root finding:** Using a two-pointer technique based on planarity, we find a safe root vertex in $O(n)$ time, where n is the number of vertices in the face. This ensures that the building cost per face can be amortized.

4. **Valid generating set (VGS):** We maintain a set of generating chords whose flip will not create a multi-edge. When flipping a chord, only four border chords change their opposite pairs; we update the VGS in constant time using linked lists.

5. **Global chord set with dynamic push/pop:** A global hash set stores all chords currently on the exploration path. This allows $O(1)$ conflict detection and supports backtracking in the recursive DFS.

6. **Pruning strategy:** If a blocking chord (i.e., a chord that does not have the root vertex as an endpoint) conflicts with a chord from a previous face, all descendant triangulations are invalid. We prune such branches safely, without losing any valid triangulation.

Our algorithm achieves the same optimal complexity as the triconnected case while handling the strictly larger class of biconnected graphs.

1.7 Comparison with Existing Algorithms

Table 1.1 summarises the differences between our algorithm and previous work.

Table 1.1: Comparison of triangulation enumeration algorithms.

Algorithm	**Graph Class**	**Time per triangulation**	**Space**
Reverse search [**?**]	General	$O(n^2)$ or worse	$O(n)$
Li–Nakano [**?**]	Biconnected	$O(r^2 n)$	—
Nakano (improved)	Biconnected	$O(rn)$	—
Parvez et al. [**?**]	Triconnected	$O(1)$ amortized	$O(n)$
This work	**Biconnected**	$\mathbf{O(1)}$ **amortized**	$\mathbf{O(n)}$

Our algorithm is the first to achieve constant amortised time for the class of biconnected plane graphs, which includes many practical instances.

1.8 Thesis Organization

The remainder of this thesis is structured as follows.

- **Chapter 2** introduces the necessary graph-theoretic concepts: connectivity, planarity, cycles, chords, triangulations, flipping operations, and genealogical trees. Special emphasis is given to separation pairs and the distinction between triconnected and biconnected graphs.
- **Chapter 3** describes the baseline algorithm for generating all triangulations of a single cycle (the Parvez–Rahman–Nakano algorithm) in detail. We present the definitions of blocking chords, generating chords, and leftmost blocking chords, and prove uniqueness.
- **Chapter 4** clarifies the distinction between our naming of the generating set and that of the original paper, and proves the condition for flipping generating chords.
- **Chapter 5** explains why the original algorithm fails for biconnected graphs due to separation pairs and the multi-edge problem.
- **Chapter 6** introduces our pruning strategy based on the persistence of chords that do not involve the root vertex.
- **Chapter 7** presents the multi-layer DFS that processes faces one by one, maintaining a global chord set and backtracking.
- **Chapter 8** addresses the problem of generating chords themselves causing multi-edges and introduces the concept of safe root vertices.
- **Chapter 9** proves a lower bound of $\Omega(n)$ triangulations per face, which is essential for amortised analysis.
- **Chapter 10** gives an $O(n)$ two-pointer algorithm for finding a safe root vertex.
- **Chapter 11** describes the valid generating set (VGS) and how to update it in constant time during flips.
- **Chapter 12** presents the complete algorithm in pseudocode and proves its correctness.
- **Chapter 13** analyses the time and space complexity, proving $O(1)$ amortised time per triangulation and $O(n)$ space.
- **Chapter 14** provides experimental validation on 35 test cases, including cycles, grids, and complex graphs, confirming the theoretical bounds.

- **Chapter 15** discusses why the algorithm cannot be extended to 1-connected graphs in general, giving examples where safe roots do not exist.
- **Chapter 16** outlines future work, including extensions to 1-connected graphs and 1-planar graphs.
- **Chapter 17** concludes the thesis with a summary of contributions and final remarks.

Chapter 2

Preliminaries

In this chapter, we define the basic concepts used throughout the thesis. We assume the reader is familiar with elementary graph theory, but we provide precise definitions to avoid ambiguity. Special emphasis is given to notions that are essential for biconnected plane graphs, such as separation pairs and separation vertices, which do not appear in the triconnected case.

2.1 Basic Graph Definitions

Let $G = (V, E)$ be a connected simple graph with vertex set V and edge set E. Each edge $e \in E$ is an unordered pair $\{u, v\}$ with $u, v \in V$ and $u \neq v$. We denote an edge between u and v by (u, v) or (v, u).

The **degree** of a vertex v is the number of edges incident to v. A graph is **simple** if it has no loops or multiple edges.

The **connectivity** $\kappa(G)$ of a graph G is the minimum number of vertices whose removal disconnects the graph (or reduces it to a single vertex). A graph is k**-connected** if $\kappa(G) \geq k$. Equivalently, removing fewer than k vertices leaves the graph connected.

In particular:

- A graph is **2-connected** (or **biconnected**) if it has no articulation vertices (cut vertices) and remains connected after the removal of any single vertex.
- A graph is **3-connected** (or **triconnected**) if it remains connected after the removal of any two vertices.

Definition 2.1 (Separation pair)**.** In a biconnected graph, a pair of vertices $\{a, b\}$ is called a **separation pair** if removing a and b disconnects the graph into at least two components. A biconnected graph that contains a separation pair is not triconnected.

Definition 2.2 (Separation vertex)**.** In a 1-connected graph, a vertex v is a **separation vertex** (or **cut vertex**) if removing v disconnects the graph. A graph is biconnected if and only if it has no separation vertices.

The distinction between biconnected and triconnected graphs is crucial: triconnected graphs have no separation pairs, while biconnected graphs may have them. The presence of separation pairs is the main obstacle when extending triangulation generation algorithms from triconnected to biconnected graphs.

2.2 Planar and Plane Graphs

A graph G is **planar** if it can be drawn in the plane without any edge crossings. Such a drawing is called an **embedding**. A **plane graph** is a planar graph together with a fixed embedding in the plane.

A plane graph divides the plane into connected regions called **faces**. The unbounded region is the **outer face**; all other faces are **inner faces**. Each face is bounded by a closed walk along the edges of the graph.

If every face boundary contains exactly three edges, the plane graph is called a **plane triangulation**. In this thesis, edges are not required to be straight lines; triangulation is understood combinatorially.

2.3 Separation Pairs and Separation Vertices

We now discuss separation pairs in more detail, as they are central to the biconnected case.

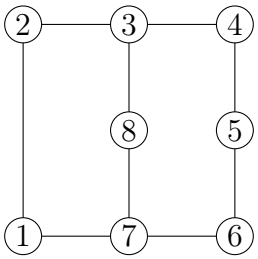


Figure 2.1: A biconnected plane graph. Vertices 3 and 7 form a separation pair.

Figure 2.1 shows a biconnected plane graph with eight vertices. Removing vertices 3 and 7 disconnects the remaining vertices into three components: $\{1, 2\}$, $\{4, 5, 6\}$, and $\{8\}$. Therefore, $(3, 7)$ is a separation pair.

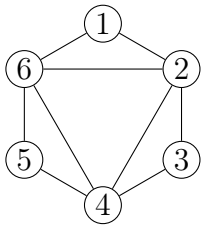
Remark 2.1. In a biconnected plane graph, a separation pair (a, b) implies that the graph can be split into two subgraphs that share only a and b. Consequently, there exist two

faces (one on each side of the pair) that can potentially contain the same chord (a, b), leading to a multi-edge if both faces are triangulated independently.

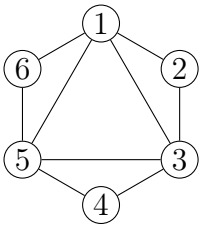
2.4 Faces, Cycles, and Chords

A **cycle** C in a graph is a sequence of distinct vertices $v_1, v_2, \ldots, v_k$ such that $(v_i, v_{i+1}) \in E$ for $1 \leq i < k$ and $(v_k, v_1) \in E$. In a plane graph, each face corresponds to a cycle that bounds it (taking the outer face as a cycle as well).

Let $C = \langle v_1, v_2, \ldots, v_n \rangle$ be a cycle, listed in counterclockwise order. An edge (v_i, v_j) is called a **chord** of C if it is not an edge of C and its interior lies inside the face bounded by C. A chord (v_i, v_j) divides the cycle into two smaller cycles: $v_i, v_{i+1}, \ldots, v_j$ and $v_j, v_{j+1}, \ldots, v_i$.



(a) A triangulation of a hexagon.



(b) Another triangulation of the same hexagon.

Figure 2.2: Two different triangulations of a cycle with six vertices.

2.5 Triangulation of a Cycle

A **triangulation** of a cycle C is a maximal set of non-intersecting chords that partition the interior of C into triangles. The sides of the triangles are either chords or edges of C. Figure 2.2 shows two triangulations of a hexagon.

Equivalently, a triangulation of a cycle with n vertices contains exactly $n - 3$ chords. This follows from Euler's formula applied to the planar graph formed by the cycle and its chords.

2.6 Labeled vs. Unlabeled Triangulations

If the vertices of the cycle are numbered sequentially from v_1 to v_n, then a triangulation is called a **labeled triangulation**. The order of vertices matters: two triangulations that differ only by a rotation or reflection are considered distinct if the labels are different.

If the vertices are not numbered, the triangulations are called **unlabeled**. In this case, rotationally equivalent or mirror-image triangulations are considered the same. In

this thesis, we work exclusively with **labeled triangulations** unless stated otherwise.

2.7 Flipping Operation

A fundamental operation for transforming one triangulation into another is the **flip**. Consider two adjacent triangles that share a chord and together form a quadrilateral. Removing the common chord and adding the opposite diagonal produces a new triangulation.

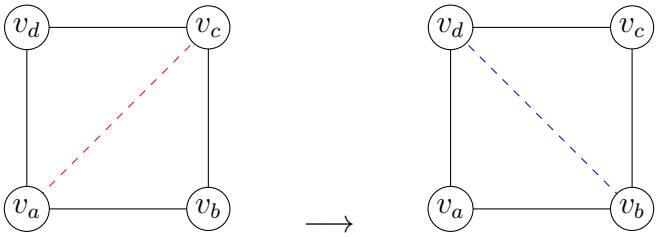


Figure 2.3: Flipping the diagonal (v_a, v_c) to (v_b, v_d) in a quadrilateral.

Formally, let (v_i, v_j) be a chord that belongs to two triangles (v_i, v_j, v_p) and (v_i, v_j, v_q) on opposite sides. These four vertices form a quadrilateral v_p, v_i, v_q, v_j in clockwise order. Flipping (v_i, v_j) means removing it and adding the opposite chord (v_p, v_q). The resulting graph is again a triangulation of the same cycle.

2.8 Visibility, Blocking Chords, and Generating Chords

Fix a cycle $C = \langle v_1, v_2, \ldots, v_n \rangle$ and choose a vertex, say v_1, as the **root vertex**. In a triangulation T of C, we say that a vertex v_j is **visible** from v_1 if the chord (v_1, v_j) belongs to T.

In the root triangulation (the fan from v_1), every vertex except the neighbours v_2 and v_n is visible from v_1. In other triangulations, some vertices may be hidden behind **blocking chords**.

Definition 2.3 (Blocking chord)**.** A chord (v_i, v_j) of a triangulation T is called a **blocking chord** if:

1. Neither v_i nor v_j is the root vertex v_1, and
2. The chord (v_i, v_j) is visible from v_1 in the sense that there is a triangle (v_1, v_i, v_j) in T.

Equivalently, a blocking chord is a chord that does not have v_1 as an endpoint but is directly connected to v_1 via triangles.

Blocking chords prevent some vertices from being visible from the root. Flipping a blocking chord replaces it with a chord that is incident to the root, thereby increasing the number of visible vertices.

Definition 2.4 (Generating chord)**.** In a triangulation T of a cycle with root vertex v_1, a chord (v_1, v_j) is called a **generating chord** if flipping it produces a new triangulation T' whose leftmost blocking chord is exactly the newly added chord.

The precise condition for a chord to be generating involves the leftmost blocking chord, which we define next.

2.9 Leftmost Blocking Chord

Among the blocking chords of a triangulation, we single out the one that is "leftmost" with respect to the cyclic order around the root.

Definition 2.5 (Leftmost blocking chord)**.** Let T be a triangulation of a cycle $C = \langle v_1, v_2, \ldots, v_n \rangle$ with root vertex v_1. Let $(v_b, v_{b'})$ be a blocking chord of T with $b < b'$. It is the **leftmost blocking chord** if no other blocking chord (v_i, v_j) satisfies $j < b'$ (i.e., its larger index is smaller than b'). In other words, among all blocking chords, the one with the smallest value of the larger endpoint is the leftmost.

Remark 2.2. The term "leftmost" comes from drawing the cycle with v_1 at the top and the remaining vertices arranged clockwise around it. Blocking chords then appear as chords that "block" the view to vertices on the left side.

We now prove that the leftmost blocking chord is unique. The following argument is taken from the author's original writing, polished for clarity.

Lemma 2.1 (Uniqueness of the leftmost blocking chord)**.** *In any triangulation T of a cycle $C = \langle v_1, v_2, \ldots, v_n \rangle$ that is not the root triangulation, there is exactly one leftmost blocking chord.*

Proof. Suppose, for contradiction, that there exist two distinct blocking chords that both qualify as leftmost. Let the two chords be $(v_b, v_{b'})$ and $(v_c, v_{c'})$ with $b < b'$ and $c < c'$. By definition of leftmost, we must have $b' = c'$ (otherwise the one with the smaller larger endpoint would be unique). Hence both chords share the same larger endpoint, say $v_{b'} = v_{c'} = v_k$.

Since both chords are blocking, each forms a triangle with the root vertex v_1. Thus we have triangles (v_1, v_b, v_k) and (v_1, v_c, v_k). The vertices v_b and v_c lie on the same side of the chord (v_1, v_k). Without loss of generality, assume $b < c$.

Consider the quadrilateral formed by v_1, v_b, v_k, v_c. Because the triangulation is planar and all faces are triangles, the chord (v_b, v_k) and (v_c, v_k) cannot both be present without

crossing. In fact, only one of them can be a chord; the other must be replaced by the opposite diagonal of the quadrilateral. However, if both are present, they would create a face with four vertices, contradicting that the triangulation is maximal. Therefore, it is impossible to have two distinct blocking chords with the same larger endpoint. Hence the leftmost blocking chord is unique. □

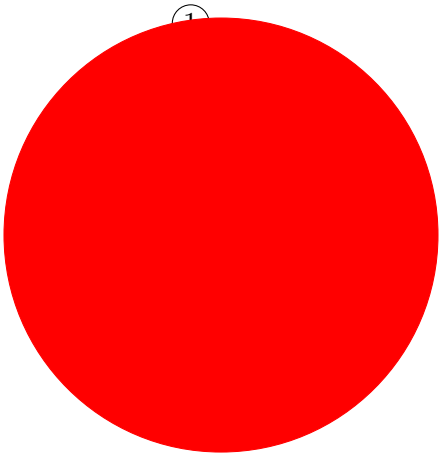


Figure 2.4: Illustration of the leftmost blocking chord. Here (v_4, v_6) is the leftmost blocking chord because no other blocking chord has a larger endpoint smaller than 6.

2.10 Genealogical Tree of Triangulations

The parent–child relationship defined by flipping the leftmost blocking chord yields a rooted tree structure on the set of all triangulations of a cycle.

Definition 2.6 (Genealogical tree)**.** Let C be a cycle with a fixed root vertex v_1. The **genealogical tree** $\mathcal{T}(C)$ of triangulations of C is defined as follows:

- The root is the fan triangulation where all chords are incident to v_1.
- For any non-root triangulation T, its parent $P(T)$ is obtained by flipping the leftmost blocking chord of T.
- Conversely, a triangulation T is a child of T' if $T' = P(T)$.

The genealogical tree has the following properties (proved in Chapter **??**):

- Every triangulation appears exactly once.
- The root has $n - 3$ children (all chords are generating).
- The depth of the tree is at most $n - 2$.
- Flipping a generating chord takes constant time.

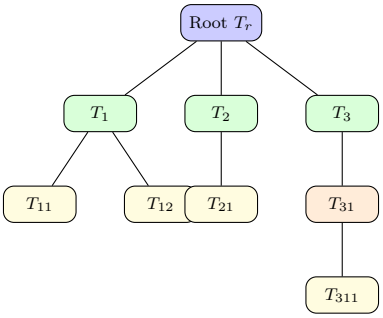


Figure 2.5: A schematic genealogical tree for a cycle. Each node represents a triangulation, and edges correspond to flipping a generating chord.

This tree structure is the basis of the algorithm for generating all triangulations of a cycle, which we recall in Chapter **??**.

2.11 Summary

We have introduced the essential graph-theoretic concepts: connectivity, planarity, faces, cycles, chords, triangulations, and the flipping operation. We defined blocking chords, generating chords, and the leftmost blocking chord, and proved its uniqueness. The genealogical tree provides a canonical parent–child relationship among triangulations of a cycle.

For biconnected graphs, we highlighted the significance of separation pairs, which are the main obstacle to extending the triconnected algorithm. These concepts will be used throughout the remainder of the thesis.